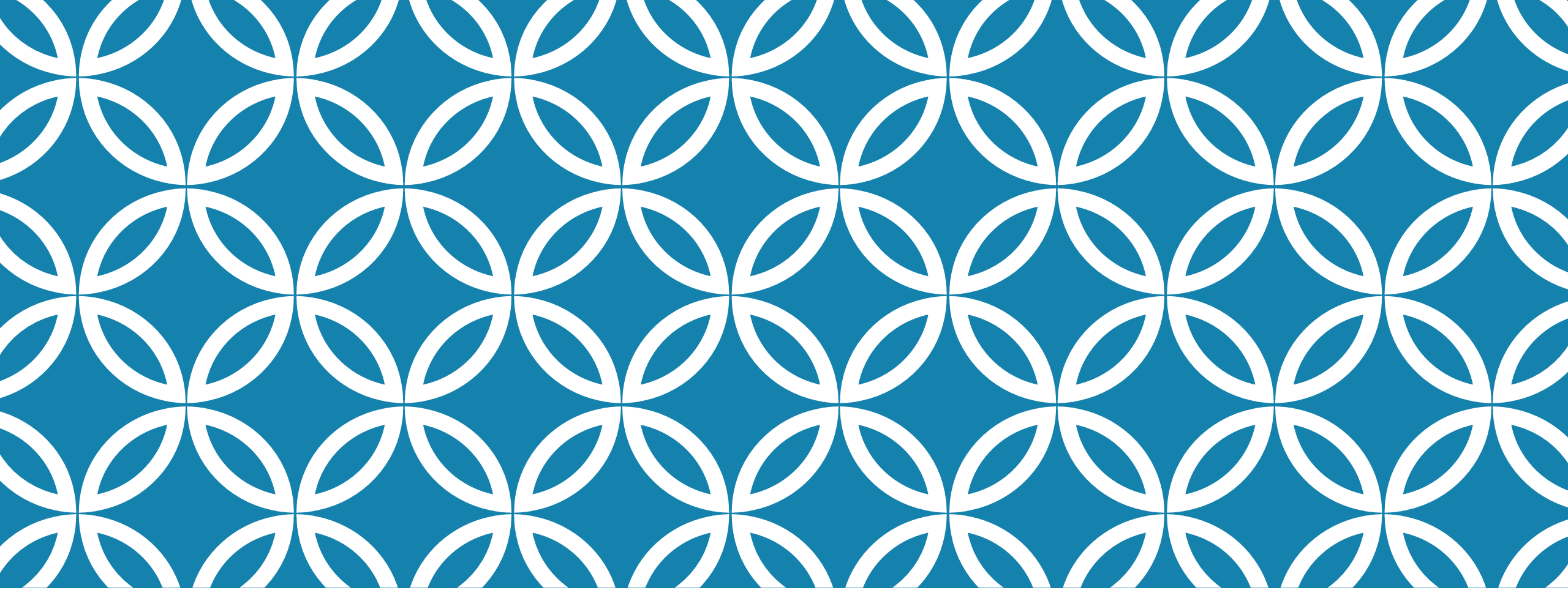

STATIC PROGRAM ANALYSIS

Lecture 10 – Data-flow framework in WALA

WALA SET UP

Pull from static-program-analysis

Files to focus on

- src/dataflow package
  - IntraprocReachingDefs.java
  - IntraporcReachingDefsDriver.java
- test/dataflow package
  - HelloWorldFields.java
- scope
  - helloworldFields.txt (change to the correct path to application src/binaries on your machine)

Run gradle to compile

Run IntraporcsReachingDefsDriver with the following parameters

- -scopeFile scope/helloworldFields.txt -entryClass dataflow/HelloWorldFields -method main

ANALYSIS DATA-STRUCTURE SET UP

1. Putstatic instructions correspond to definitions
2. Analysis first counts how many putfield instructions in the method: k
   - kept in putInstrs as actual count in the method
3. Uses a bit-vector data-structure where value a bit at index m corresponds to putstatic instruction at instruction count m ($1 \leq m \leq k$)
   - if the bit at index m set to 0, then putstatic with count m is not present
   - if the bit at index m set to 1, then putstatic with count m is present
4. Creates a bitvector for each filed, for x and for y
5. Sets bits where a field is defined (for putstatic instructions) to 1 (for the kill part of the transfer function)
6. Sets data-structure for IN/OUT is a bitvector of size k
   - IN(stmt) = (0,1,0,0) means that instruction at count 1 reached that statement (instruction should be putstatic)
   - toString of (0,1,0,0) is {2}, toString of (0,1,0,1) is {2,4}

Efficient modeling of sets as bitvecotrs. Easy/fast ∪ and ∩ operations

TRANSFER FUNCTION

TransferFunctions **implements** ITransferFunctionProvider

if instruction is putstatic

- resolves its field
- gets field's kill function
- creates this instruction gen bitvector
- creates an operator object with kill and gen pair
  - **return new** BitVectorKillGen(kill, gen);
  - used by framework to update IN value into OUT value

If instruction is not putstatic

- create an identity operator **return** BitVectorIdentity.*instance*();

Identifies how to merge flows BitVectorUnion.*instance*();

UNDERSTANDING FRAMEWORK

Instantiates BitVectorFramework

- IN/OUT values are bit-vectors

```
framework = new BitVectorFramework<IExplodedBasicBlock, Integer>(ecfg, new TransferFunctions(), putInstrNumbering);
```

values to use in IN/OUT results

describes how to change flow
how to merge flows

Creates a solver, and invokes solve method on it.

OTHER ANALYSIS

Let's try to implement another classical analysis